

人工编写源码:

```
def bubble_sort(arr):
    """
    对列表 arr 进行原地冒泡排序（从小到大）
    """
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        # 内层循环：每一趟把当前未排序部分的最大值“冒泡”到末尾
        for j in range(0, n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # 如果本趟无交换，说明已整体有序，提前结束
        if not swapped:
            break

if __name__ == "__main__":
    raw = input("请输入要排序的整数列表（用空格分隔）：")
    arr = [int(x) for x in raw.split()] if raw.strip() else []
    print("原列表：", arr)

    bubble_sort(arr)

    print("排序后：", arr)
```

反编译复原源码:

```
def bubble_sort(arr):
    """
    对列表 arr 进行原地冒泡排序（从小到大）
    """
    n = len(arr)
    for i in range(n - 1):
        swapped = False

        for j in range(0, n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
            if not swapped:
                break

if __name__ == "__main__":
    raw = input("请输入要排序的整数列表（用空格分隔）：")
    arr = [int(x) for x in raw.split()] if raw.strip() else []
    print("原列表：", arr)
    bubble_sort(arr)
    print("排序后：", arr)
```

等价性论证：

接口：

都通过 `if __name__ == "__main__":` 处理输入，调用排序函数、输出结果。

都定义了 `bubble_sort(arr)` 作为冒泡排序算法的输入，参数和返回值形式一致

算法逻辑：

步骤一致，都为：

1. 取列表长度 `n = len(arr)`
2. 开始外层循环 `for i in range(n - 1):`
3. 每一次外层开始时都设置 `swapped = False`
4. 随后开始内层循环 `for j in range(0, n - 1 - i):`
5. 比较相邻元素并决定是否交换位置
6. 一轮结束后如果没发生过交换则提前退出 `if not swapped:
break`

所以不管是从控制流（两层循环和提前退出）还是数据操作（相邻元素比较并交换）的角度看，两个版本都是等价的。

人工编写优势总结：

1. 注释更丰富且明确：人工编写的源码中有详细的注释，可读性更强。普通注释在编译过程中是虚拟机直接忽视的，字节码反编译成的 python 源代码只注重程序功能的实现而不是可读性。
2. 结构层次清晰：人工编写版本将算法逻辑分为清晰的几个步骤（初始化准备，外层循环分轮次，内层循环实现冒泡，提前结束判定）。而反编译版本只注重实现逻辑而忽略体现代码逻辑的层次结构，没有明确的内外循环逻辑体现出来，缺少“美感”。

AI 辅助声明表：

1. 使用场景：初始算法选型咨询
2. 提示词原文：

我是一名大四的软件工程专业学生，附件中是我们的函数式编程与编译原理课程的作业，请详细浏览作业要求，帮我总结以下几个点： 1. 作业目标是什么 2. 作业涉及的知识点有哪些 3. 两个任务的内容与要求是什么 4. 两个任务的大致程序编写流程是什么样的 5. 附录中的大模型使用声明表具体是如何要求的

那么对于初学者而言，这两个任务哪一个是比较容易快速上手，能在短时间内完成源码编写以及报告撰写的

我大概明白了，任务二的要求就是要：首先编写一个复杂度为 $O(n^2)$ 的 python 程序用反汇编工具将程序转化为汇编语言根据转化出的汇编代码手写等价的汇编代码将汇编代码通过编译工具转化成高级语言并对比二者的不同优劣等对吗

好的，那么接下来我要编写一个简单的复杂度为 $O(n^2)$ 的算法程序，请问有什么推荐的类似算法吗

附件中是我的 python 程序反汇编后生成的字节码内容，接下来我要逐函数对这些字节码进行注释，请按照函数功能帮我将这些字节码进行分块，方便我定位函数。